

Architektura Holo-Haptyczna: Przełomowy Ekosystem WebGPU i SDF w Paradygmacie Maksymalizmu Taktylnego

Wstęp: Konieczność Neuroergonomiczna i Zmierzch Płaskiego Paradygmatu

Współczesna inżynieria interfejsów użytkownika znajduje się w punkcie krytycznej transformacji ewolucyjnej. Przez ponad dekadę dominującym dogmatem projektowym pozostawał skrajnie płaski, sterylny design, zoptymalizowany pod kątem responsywności i minimalnego obciążenia przeglądarek. Jednakże podejście to całkowicie zignorowało fundamentalną prawdę poznawczą: ludzki aparat wzrokowy ewoluował w fizycznym, trójwymiarowym świecie, zdominowanym przez skomplikowaną grę światła, cienia, głębi i tekstury. Przejście od płaskich interfejsów do nurtu określanego jako maksymalizm taktylny (tactile maximalism) nie jest jedynie przejściową modą estetyczną, lecz absolutną koniecznością neuroergonomiczną. Użytkownicy, coraz bardziej zmęczeni jednorodnością i syntetycznością środowisk cyfrowych, poszukują interfejsów, które angażują zmysły, oferują poczucie fizycznej wagi i namacalności.

W dobie maksymalizmu taktylnego cień w środowisku cyfrowym przestał pełnić funkcję wyłącznie dekoracyjną. Stał się kluczowym rusztowaniem strukturalnym, które zarządza uwagą użytkownika, buduje hierarchię informacji, redukuje zmęczenie poznawcze i definiuje fizyczną spójność interfejsu. Przyciski muszą wyglądać tak, jakby stawiały fizyczny opór, karty informacyjne powinny zachowywać się jak obiekty nałożone na siebie w osi Z, a ikony muszą posiadać odczuwalną teksturę i ciężar. Niestety, narzędzia i standardy, którymi dysponują obecnie inżynierowie front-endu — w szczególności kaskadowe arkusze stylów (CSS) i tradycyjny Document Object Model (DOM) — są strukturalnie niezdolne do sprostanienia tym wymaganiom bez drastycznych kompromisów wydajnościowych.

W niniejszym dokumencie przeprowadzona zostanie wyczerpująca analiza krytycznych braków w dzisiejszych systemach renderowania interfejsów użytkownika. Zostaną zidentyfikowane kluczowe luki technologiczne oraz bezpośrednie bariery dla wydajności i satysfakcji użytkownika. Następnie zaprezentowane zostanie przełomowe, futurystyczne rozwiązanie — architektura Holo-Haptic WebGPU Spatial Framework (HHWSF) — która całkowicie redefiniuje sposób, w jaki przeglądarki komunikują się z procesorem graficznym (GPU) i urządzeniami peryferyjnymi, pozwalając na wyznaczenie nowych, nieosiągalnych dotąd standardów rynkowych.

Analiza Krytyczna: Identyfikacja Brakujących Elementów i Ograniczeń Architektonicznych

Obecna architektura interfejsów opartych na języku HTML i CSS wykazuje szereg

fundamentalnych ograniczeń, które uniemożliwiają stworzenie prawdziwie immersyjnego doświadczenia przestrzennego. System ten traktuje renderowanie każdej warstwy jako izolowane zadanie kompozytora, ignorując relacje przestrzenne i zjawiska fizyczne, które występują między obiektami. Poniżej zidentyfikowano najważniejsze braki i zaproponowano kierunkowe, innowacyjne koncepcje ich rozwiązania.

Ślepota Środowiskowa i Izolacja Kontekstowa

Pierwszym i najbardziej rażącym brakiem dzisiejszych systemów jest całkowita izolacja interfejsu od fizycznego środowiska użytkownika. Ekran urządzenia stanowi hermetyczną barierę. Interfejs użytkownika generuje własne, statyczne oświetlenie, niezależnie od tego, czy urządzenie znajduje się w całkowitej ciemności, czy w pełnym słońcu. Zjawisko to można określić mianem "ślepoty środowiskowej". Choć nowoczesne smartfony i laptopy są wyposażone w precyzyjne czujniki oświetlenia (Ambient Light Sensors) oraz żyroskopy, dane te rzadko są wykorzystywane do kształtowania interfejsu w czasie rzeczywistym.

Rozwiązaniem tego braku jest integracja sensoryki środowiskowej bezpośrednio z silnikiem renderującym. Zastosowanie Generic Sensor API, w tym AmbientLightSensor oraz interfejsów orientacji urządzenia (DeviceOrientation), pozwala na przechwytywanie wektorów natężenia i kierunku światła ze świata rzeczywistego. Przełomowym rozwiązaniem jest dynamiczne mapowanie tych danych jako zmiennych jednorodnych (uniforms) przesyłanych do procesora graficznego, co pozwala na rzutowanie cyfrowych cieni dokładnie w tym samym kierunku, w którym padają cienie fizyczne w pokoju użytkownika.

Brak Ujednoliconej Fizyki Oświetleniowej (Paradygmat Pudełkowy)

Tradycyjne podejście do cieniowania w CSS (np. box-shadow) opiera się na bezrefleksyjnym deklarowaniu rzutowania rozmytego prostokąta lub okręgu pod każdym elementem z osobna. System ten ignoruje fakt, że interfejs użytkownika powinien być metafizyczną przestrzenią podlegającą jednolitej fizyce oświetleniowej. W standardowym CSS nie istnieje pojęcie globalnego źródła światła, co prowadzi do absurdów optycznych, gdzie różne elementy na tym samym ekranie rzucają cienie w sprzecznych kierunkach. Ponadto standardowe narzędzia renderowania nie potrafią poprawnie generować półcieni ani zachowań światła otoczenia. Rozwiązaniem tej luki jest porzucenie modelu box-shadow na rzecz matematyki Pól Odległości (Signed Distance Fields - SDF) renderowanych za pomocą nowoczesnego interfejsu WebGPU. SDF pozwala na matematyczne opisanie dowolnego kształtu poprzez funkcję, która zwraca odległość danego punktu przestrzeni od najbliższej krawędzi tego kształtu. Pozwala to na zastosowanie zaawansowanych technik takich jak raymarching (śledzenie promieni), gdzie promienie światła są symulowane w czasie rzeczywistym, odbijając się i uginając na krawędziach elementów UI, tworząc idealne, miękkie cienie (soft shadows) zgodne z prawami fizyki.

Dysproporcja Sensoryczna i Odcięcie Haptyczne

Trzecim poważnym brakiem jest brak zgodności pomiędzy wizualną reprezentacją ciężaru i głębi elementu a jego faktyczną odpowiedzią dotykową. Maksymalizm taktylny operuje pojęciami wypukłości (embossing) oraz wklęsnięcia (debossing), sugerując użytkownikowi fizyczny opór interfejsu. Niestety, interakcja z tymi pięknie wyrenderowanymi strukturami zazwyczaj kończy się uderzeniem palca w twardą, gładką szybę urządzenia, co prowadzi do

dysonansu poznawczego. Choć proste API wibracji istnieje, jest rzadko sprzężone z wizualną głębią na osi Z.

Rozwiązaniem jest budowa Haptograficznej Matrycy Osi Z (Haptographic Z-Axis Matrix). Poprzez wykorzystanie Vibration API w przeglądarkach oraz niskopoziomowych sterowników urządzeń piezoelektrycznych, możliwe jest dynamiczne generowanie profili wibracyjnych, które matematycznie odpowiadają wyliczonej wcześniej elewacji elementu na osi Z. Przełom polega na tym, że wysokość cienia renderowanego przez WebGPU staje się bezpośrednią zmienną zasilającą siłę aktuatora piezoelektrycznego, tworząc system, w którym cyfrowe wzniesienia można "poczuć".

Przełomowa Koncepcja Futurystyczna: Sentient Neuro-Spatial Mesh

Aby nie tylko wypełnić powyższe luki, ale też radykalnie przeprojektować system i wyprzedzić konkurencję, należy zintegrować wymienione technologie w jedną, odważną i niekonwencjonalną usługę. Koncepcją tą jest "Sentient Neuro-Spatial Mesh" (Świadoma Sieć Neuro-Przestrzenna) zaimplementowana w ramach Holo-Haptic WebGPU Spatial Framework (HHWSF).

Jest to środowisko, w którym interfejs użytkownika staje się żywym organizmem. Interfejs ten odchodzi od koncepcji stron i widoków na rzecz topologii przestrzennej. W "idealnej wersji" tego systemu, interfejs wyczuwa użytkownika przed fizycznym kontaktem (za pomocą danych z mikrokamery i analizy spojrzenia) oraz adaptuje swój własny ciężar wizualny i haptyczny do stanu emocjonalnego użytkownika. Jeśli sensory biometryczne lub analiza rytmu interakcji wykażą, że użytkownik jest pod wpływem stresu lub presji czasu, interfejs automatycznie redukuje głębię cieni, wygładza krawędzie SDF (zwiększając promień przenikania za pomocą operacji matematycznych takich jak funkcja smin) i emituje uspokajającą poświatę (Emissive Neon Glow). Równocześnie wibracje piezoelektryczne przechodzą z ostrych, klikających impulsów do miękkich, niskoczęstotliwościowych fal.

Z technologicznego punktu widzenia, system ten operuje z pominięciem tradycyjnego drzewa rysowania DOM. Elementy nawigacyjne, karty i przyciski są definiowane semantycznie w HTML dla celów ułatwień dostępu (Accessibility), jednak ich wizualna powłoka jest wymazywana i delegowana do procesora graficznego poprzez połączenie CSS Houdini PaintWorklet oraz niewidzialnego płótna WebGPU Canvas (OffscreenCanvas). Kod shaderów pisany w języku WGSL (WebGPU Shading Language) na bieżąco oblicza oświetlenie przestrzenne, mieszając cienie kameleonowe, które absorbują kolory podłoża. Zapewnia to renderowanie z nieskończoną rozdzielczością (ze względu na matematyczną naturę SDF) przy ułamku kosztu energetycznego standardowych filtrów SVG czy rozmyć z kaskadowych arkuszy stylów.

Wizualizacja: Relacyjna Mapa Struktur i Architektury

Aby w pełni zrozumieć ewolucję systemu, konieczne jest zestawienie istniejących, przestarzałych struktur z nowoczesnymi komponentami wynikającymi z integracji brakujących elementów. Poniższa tabela stanowi mapę semantyczną ukazującą tę transformację.

Komponent Systemowy	Obecny Stan (Architektura Pudełkowa DOM)	Identyfikacja Braku i Źródło Problemu	Zoptymalizowany Stan Przyszły (Architektura HHWSF)
Generowanie Światłocienia i Głębi	Rasteryzacja i aplikacja filtru rozmycia gaussowskiego na prostokątny kontur (właściwość box-shadow).	Brak spójnego źródła światła, brak poprawnego przenikania cieni (ambient occlusion), drastyczne spadki wydajności przy animacji rozmycia na osi Z.	Ewaluacja matematyczna odległości (SDF) w ujednoliconym środowisku WebGPU; implementacja algorytmu raymarching 2D i miękkich cieni (Soft Shadows).
Responsywność Środowiskowa	Statyczny widok. Jedyną zmienną jest flaga systemowa prefers-color-scheme (jasny/ciemny).	Interfejs jest odłączony od warunków panujących w pomieszczeniu. Cienie są statyczne względem ekranu.	Pełna integracja z Generic Sensor API. AmbientLightSensor dyktuje intensywność globalnego światła, a DeviceOrientation definiuje wektor kierunkowy dla shadera WGSL.
Spójność Geometrii i Masek	Właściwość clip-path tworzy brutalne odcięcie matematyczne przestrzeni elementu, bezpowrotnie niszcząc zewnętrzne cienie i rozmycia kontenera.	Konflikt w silniku kompozytowym przeglądarki między modelem pudełkowym a wektorowym maskowaniem ścieżek.	Użycie funkcji boolowskich SDF (np. max(d1, -d2) dla wycinania) pozwala na bezkolizyjne łączenie masek z globalnym silnikiem światła bez jakichkolwiek ucięć.
Reakcja Haptyczna na Interakcję	Opieranie się na domyślnych, zero-jedynkowych ustawieniach wibracji systemu operacyjnego.	Brak sprzężenia pomiędzy wizualną reprezentacją wagi i głębi (np. efekt wytlóczenia) a odczuciem fizycznego oporu.	Haptograficzna Matryca osi Z. Natężenie i krzywa wibracji wyliczane są dynamicznie przez JavaScript na podstawie tokenu elewacji elementu z systemu designu.
Postępowanie z Tłem w Dark Mode	Odrzucenie cieni jako "niewidocznych" na ciemnym tle, stosowanie płaskiej, achromatycznej czerni.	Interfejs traci hierarchię wizualną i staje się nieczytelną, płaską plamą (achromatyczne kłamstwo).	Wdrożenie techniki Luminance Step-Up (Kaskadowe Stopniowanie Luminancji) oraz renderowanie cieni emisyjnych (Emissive Neon Glow) dla elementów

Komponent Systemowy	Obecny Stan (Architektura Pudełkowa DOM)	Identyfikacja Braku i Źródło Problemu	Zoptymalizowany Stan Przyszły (Architektura HHWSF)
			interaktywnych.

Schemat Przejścia i Wizualny Przepływ Naprawionego Systemu

Architektura docelowego, naprawionego systemu przypomina bardziej silnik gry wideo połączony z sensoryką sprzętową niż tradycyjną stronę internetową. Poniżej przedstawiono zoptymalizowany przepływ danych (Data Flow), ilustrujący drogę od środowiska fizycznego do wyrenderowanego piksela i sprzężenia zwrotnego.

Warstwa Architektoniczna	Przepływ Procesów i Komunikacja Modułów	Narzędzia i Technologie
Wejście Środowiskowe (Hardware)	Pozyskiwanie surowych danych w czasie rzeczywistym. Sensor światła zbiera luks (lux), żyroskop ustala macierz obrotu, a ekran dotykowy rejestruje wektory nacisku.	AmbientLightSensor, DeviceOrientation, Touch Events API.
State Management (Logika Biznesowa)	Przetwarzanie i normalizacja szumu z sensorów. Mapowanie elewacji z osi Z na tokeny z systemu designu (np. Z-0 do Z-3). Wyliczanie pozycji wirtualnego światła w przestrzeni NDC (Normalized Device Coordinates).	React, Next.js, Redux, Math.js.
Houdini Bridge (Integrator DOM)	Powiązanie semantycznych elementów HTML z silnikiem renderującym. Przekazanie zmiennych konfiguracyjnych poprzez CSS Custom Properties (np. --light-pos-x) do modułu renderującego.	CSS Paint API, PaintWorklet.
WebGPU Render Pipeline (Rdzeń)	Przekazanie zmiennych do buforów (Uniform Buffers). Wywołanie Compute Shadera do zaktualizowania przestrzeni SDF. Wywołanie Fragment Shadera w języku WGSL do obliczenia oświetlenia i raymarchingu 2D.	interfejs WebGPU, WGSL, GPUCanvasContext.
Wyjście Sensoryczne (Hardware)	Zwrócenie idealnie oświetlonych pikseli na ekran urządzenia. Wyzwolenie profilu piezoelektrycznego	OffscreenCanvas, Vibration API, Piezo Actuators.

Warstwa Architektoniczna	Przepływ Procesów i Komunikacja Modułów	Narzędzia i Technologie
	adekwatnego do obliczonej wagi wyrenderowanego elementu w odpowiedzi na dotyk.	

Przejście od stanu obecnego do docelowego wymaga zaplanowania ścisłego harmonogramu transformacji, co minimalizuje ryzyko regresji w dużych systemach klasy enterprise.

Faza 1: Neutralizacja długu wydajnościowego (Miesiąc 1-2) Działania skupiają się na usunięciu najcięższych architektonicznych błędów w kaskadowych arkuszach stylów.

Refaktoryzacja kodu polega na usunięciu animacji właściwości box-shadow i zastąpieniu jej warstwową zmianą przezroczystości (opacity) na elementach pseudo. Wdrożenie systemu

Double Wrapper do omijania przycinania masek clip-path. **Faza 2: Konstrukcja warstwy integracyjnej Houdini (Miesiąc 3-4)** Zbudowanie polifilli dla przeglądarek nieobsługujących nowych standardów. Utworzenie rejestru skryptów PaintWorklet, które zaczną rysować proste generatywne cienie wokół elementów bez użycia renderera przeglądarki, korzystając z interfejsu CanvasRenderingContext2D. **Faza 3: Transpozycja środowiskowa i sprzętowa (Miesiąc 5-6)** Wdrożenie mikrousług w warstwie front-endowej monitorujących

AmbientLightSensor. Połączenie zmiennych środowiskowych z utworzonymi skryptami Houdini. Wdrożenie autorskich bibliotek dla sprzężenia haptycznego (Piezo-Haptic Engine)

sprzęgniętych ze zdarzeniami zdarzeń interfejsu. **Faza 4: Fuzja WebGPU i SDF (Miesiąc 7-8)** Ostateczna migracja z Canvas 2D API na wysokowydajne potoki WebGPU wewnątrz

środowiska Houdini. Przepisanie logiki rysującej na język WGSL, wdrażające natywny raymarching oraz wsparcie dla kameleonowego cieniowania w trybie dark mode (Luminance Step-Up). Optymalizacja pod kątem mobilnych układów GPU.

Identyfikacja Barrier Wydajnościowych i Wysoko Wpływowe Działania

Przed osiągnięciem finalnego, zoptymalizowanego środowiska opartego na WebGPU, systemy mierzą się z szeregiem barier. Przeanalizowano główne przyczyny ich powstawania i opracowano priorytetowe, punktowe działania naprawcze (Quick Wins), charakteryzujące się niewielkim nakładem pracy, lecz gigantycznym wpływem na architekturę.

Priorytet 1: Bariera Energetyczna i Przepalanie Klatek (Framerate Drops)

Przyczyna bariery: Głównym powodem drastycznego zużycia baterii oraz spadków płynności w nowoczesnych aplikacjach webowych jest niewłaściwe zarządzanie silnikiem kompozytowym przeglądarki. Deweloperzy nagminnie animują właściwości geometryczne i obciążające, takie jak box-shadow, przy zdarzeniach takich jak :hover. Zmusza to jednostkę graficzną (GPU) do ciągłego, z klatki na klatkę (120 FPS), przeliczania skomplikowanych algorytmów rozmycia gaussowskiego dla każdego piksela objętego modyfikacją. **Wysoko wpływowe działanie (The Maestro Hack):** Usunięcie błędu odbywa się poprzez radykalną zmianę paradygmatu animacji. Zamiast płynnie zmieniać parametry cienia bezpośrednio na elemencie bazowym, należy stworzyć niezależne warstwy ostateczne przy użyciu pseudoelementów (::before dla stanu

spoczynku, ::after dla stanu docelowego z potężnym, szerokim rozmyciem). Oba elementy są przygotowane z wyprzedzeniem i wyrenderowane jednorazowo. Następnie, na zdarzenie :hover, animowana jest wyłącznie ich właściwość opacity w połączeniu z dyrektywą will-change: opacity. Animacja przezroczystości realizowana jest w całości na poziomie sprzętowego kompozytora (Hardware Compositing), omijając potok repaint/reflow. Redukuje to zużycie procesora o ponad 90% i gwarantuje płynność na każdym urządzeniu.

Priorytet 2: Strukturalna Niespójność Masek Geometrii

Przyczyna bariery: W zaawansowanych interfejsach stylizowanych na estetykę sci-fi lub high-tech, często wykorzystuje się niestandardowe kształty rzeźbionych kart. Do tego celu służy właściwość CSS clip-path. Jednakże, zaaplikowanie tej właściwości na element powoduje bezwzględne, matematyczne odcięcie jakiegokolwiek grafiki (w tym tekstur, obramowań zewnętrznych i cieni rzucanych na zewnątrz) poza zadeklarowanym obrysem. Element traci całkowicie poczucie osadzenia w przestrzeni. **Wysoko wpływowe działanie (Struktura Double Wrapper):** Problem rozwiązuje się poprzez architektoniczną separację odpowiedzialności. Zamiast nakładać maskę i generować cień wewnątrz jednego węzła DOM, tworzy się strukturę podwójnej kapsuły. Zewnętrzny element-kontener (wrapper) odpowiedzialny jest wyłącznie za generowanie fizycznego cienia docelowego (korzystając np. z filtra drop-shadow, który naturalnie dopasowuje się do przezroczystości zawartości) oraz rezerwuje przestrzeń brzegową (padding). Wewnętrzny element (dziecko) otrzymuje deklarację clip-path oraz ukrywa wylanie się zawartości (overflow-hidden). To subtelne rozbitcie pozwala zachować skomplikowaną geometrię przy jednoczesnym zachowaniu epickiej wizualnej głębi i trójwymiarowości bryły.

Priorytet 3: Achromatyczne Kłamstwo w Środowiskach Dark Mode

Przyczyna bariery: Interfejsy w trybie nocnym często tracą swoją czytelność. Projektanci, opierając się na przyzwyczajeniach z jasnego trybu, po prostu odwracają paletę kolorów i próbują generować czerń w postaci rgba(0,0,0, 0.5) pod elementami na bardzo ciemnym tle. Taki cień fizycznie wtapia się w nicość, uniemożliwiając ludzkiemu oku percepcję jakiegokolwiek głębi. Brak światła i spłaszczenie topologii tworzy ogromną barierę dla nawigacji i użyteczności. **Wysoko wpływowe działanie (Kaskadowe Stopniowanie Luminancji oraz Emissive Glow):** Architektura ciemnego motywu musi zostać przepisana. Zamiast na samych cieniach, system musi oprzeć się na właściwości zdefiniowanej jako "Luminance Step-Up" (Stopniowe podnoszenie luminancji). Każdy wyższy poziom osi Z (np. najwyższe Z-3 dla wyskakujących okien dialogowych) wymaga matematycznie jaśniejszego tła bazowego (np. kolor #003737 zamiast głębokiego #001111). Dodatkowo, zamiast generować tradycyjne ciemne cienie absorbujące światło, elementy interaktywne na ciemnym tle powinny emitować poświatę (Emissive Neon Glow) jasnego, wysoko nasyconego koloru (np. fioletowego lub cyjanu) spod spodu, działając jak obiekty samoemisyjne rodem z estetyki Cyberpunk. Definiuje to nową jakość nawigacyjną i radykalnie podnosi satysfakcję estetyczną użytkownika.

Turbo-Szczegółowy Dokument Produkcyjny (Ecosystem Blueprint)

Ta sekcja stanowi kompletny, inżynierski przewodnik implementacyjny, demonstrujący krok po

kroku budowę środowiska Holo-Haptic WebGPU Spatial Framework w warunkach produkcyjnych. Przewodnik opisuje integrację z frameworkami warstwy abstrakcji układu (takimi jak React i Next.js), architekturę shaderów WebGPU do budowy pól odległości (SDF) oraz zarządzanie zmysłową odpowiedzią zwrotną.

Technika 1: Architektura i Konfiguracja Potoku WebGPU w React/Next.js

Zarządzanie stanem i życiem interfejsu w technologii React wymaga, by środowisko WebGPU było poprawnie inicjalizowane jako ukryty proces poboczny, nieblokujący głównego wątku renderowania. Poniżej zaprezentowano logikę izolującą ten proces.

Architektura opiera się na komponencie kontenerowym wstrzykującym płótno Canvas, które rezygnuje z interfejsów WebGL na rzecz nowoczesnego API asynchronicznego.

```
// lib/webgpu/SpatialUIContext.js
[span_67](start_span) [span_67](end_span) [span_68](start_span) [span_68](end_span) [span_69](start_span) [span_69](end_span)
import { useEffect, useRef } from 'react';

export function useWebGPUSpatialEngine() {
  const canvasRef = useRef(null);
  const deviceRef = useRef(null);

  useEffect(() => {
    let isMounted = true;

    async function initializeGPU() {
      if (!navigator.gpu) {
        console.warn("Brak wsparcia WebGPU. Trwa przełączanie na tryb degradacji (Fallback to CSS).");
        return;
      }

      try {
        // 1. Złożenie zapytania o niskopoziomowy dostęp do
        adaptera
        [span_70](start_span) [span_70](end_span) [span_71](start_span) [span_71](end_span)
        const adapter = await navigator.gpu.requestAdapter({
          powerPreference: "high-performance" // Wymuszenie
        dyskretne GPU dla maks. jakości
        });

        // 2. Utworzenie bezpiecznego, logicznego urządzenia z
        włączonymi rozszerzeniami
        const device = await adapter.requestDevice();
        if (!isMounted) return;
        deviceRef.current = device;
      } catch {
        // Obsługa błędów
      }
    }

    initializeGPU();
  }, []);
}
```



```

        const canvas = canvasRef.current;
        const context = canvas.getContext("webgpu");
        const format =
navigator.gpu.getPreferredCanvasFormat();

        // 3. Konfiguracja kontekstu zgodnie ze specyfikacją

con[span_64](start_span)[span_64](end_span)[span_66](start_span)[span_
66](end_span)text.configure({
            device,
            format,
            alphaMode: "premultiplied",
            usage: GPUTextureUsage.RENDER_ATTACHMENT |
GPUTextureUsage.COPY_DST // Gotowość do potoków cieniowania
        });

        // Inicjalizacja WGSL potoku Shaderów
        buildRenderPipeline(device, context, format);

        } catch (err) {
            console.error("Błąd sprzętowy środowiska WebGPU:",
err);
        }
    }

    initializeGPU();
    return () => { isMounted = false; };
},,);

    return canvasRef;
}

```

Implementacja produkcyjna wymaga nałożenia płótna w tle ekranu (globalny div z indeksem Z o wartości ujemnej), a wszystkie inne interfejsy DOM funkcjonują nad nim z ustawioną przezroczystością tła, czerpiąc światłocienie dostarczane od dołu.

Technika 2: Shader WGSL do Pól Odległości (SDF) i Raymarchingu

Sercem nowej architektury jest kod napisany w języku WGSL. Nie renderujemy tu miliardów wierzchołków z pomocą shadera vertexów (jak to ma miejsce w modelach 3D), lecz wysyłamy jeden pełnoekranowy czworokąt. To Fragment Shader bierze na siebie gigantyczny ciężar wykonania algorytmów raymarchingu 2D i obliczania wektorów oświetlenia.

Poniżej przedstawiono bazowy rdzeń produkcyjny obliczania Signed Distance Fields dla zaokrąglonych paneli interfejsu oraz miękkiego śledzenia cieni, zaimplementowany jako kod przesyłany do potoku WebGPU.

```

// shaders/spatial_ui.wgsl
[span_77](start_span)[span_77](end_span)[span_78](start_span)[span_78]

```

```
(end_span)
```

```
struct UIBuffer {  
    resolution: vec2f,  
    light_direction: vec2f, // Wektor sterowany przez żyroskop i  
    sensory sprzętowe [span_79] (start_span) [span_79] (end_span)  
    ambient_intensity: f32, // Sterowane przez Ambient Light Sensor  
    [span_81] (start_span) [span_81] (end_span)  
}
```

```
@group(0) @binding(0) var<uniform> ui_params: UIBuffer;
```

```
// 1. Definicja topologiczna kształtu panelu UI za pomocą SDF  
// Funkcja wylicza dystans punktu 'p' od brzegu zaokrąglonego  
prostokąta (karty)
```

```
fn sdRoundRect(p: vec2f, size: vec2f, radius: vec4f) -> f32 {  
    var r = radius;  
    r.x = select(r.x, r.z, p.x > 0.0);  
    r.y = select(r.y,  
[span_17] (start_span) [span_17] (end_span) [span_19] (start_span) [span_19]  
(end_span)r.w, p.x > 0.0);  
    r.x = select(r.x, r.y, p.y > 0.0);  
  
    let d = abs(p) - size + vec2f(r.x);  
    return min(max(d.x, d.y), 0.0) + length(max(d, vec2f(0.0))) - r.x;  
}
```

```
// 2. Moduł Raymarchingu do tworzenia miękkich cieni (Soft Shadows)  
[span_83] (start_span) [span_83] (end_span) [span_84] (start_span) [span_84]  
(end_span) [span_85] (start_span) [span_85] (end_span)
```

```
fn calculateSoftShadow(ro: vec2f, rd: vec2f, mint: f32, maxt: f32, k:  
f32) -> f32 {  
    var res: f32 = 1.0;  
    var t: f32 = mint;
```

```
    // Algorytm śledzenia odległości krokowego  
[span_86] (start_span) [span_86] (end_span) [span_87] (start_span) [span_87]  
(end_span)
```

```
    for(var i: u32 = 0; i < 32; i++) {  
        let p = ro + rd * t;  
        let d = sdRoundRect(p, vec2f(300.0, 200.0), vec4f(16.0)); //  
Przykładowe wymiary komponentu
```

```
        if (d < 0.001) { return 0.0; } // Uderzenie w przeszkodę  
(całkowity cień)
```

```
        // Zjawisko półcienia wyliczane matematycznie na podstawie  
kąta ominięcia
```

```

        res = min(res, k * d / t);
        t += d;
        if (t > maxt) { break; }
    }
    return clamp(res, 0.0, 1.0);
}

@fragment
fn fs_main(@builtin(position) coord: vec4f) -> @location(0) vec4f {
    // Normalizacja układu odniesienia współrzędnych
    let uv = (coord.xy * 2.0 - ui_params.resolution) /
min(ui_params.resolution.x, ui_params.resolution.y);

    // Pobranie dynamicznego wektora źródła światła
    let light_dir = normalize(ui_params.light_direction);

    // Obliczenie współczynnika miękkiego cienia z użyciem
raymarchingu [span_88](start_span)[span_88](end_span)
    let shadow_intensity = calculateSoftShadow(uv, light_dir, 0.01,
10.0, 12.0);

    // Kompozycja kolorów środowiska, w tym zasada 'Chameleon Shadows'
(Cienie Kameleonowe)
    let surface_color = vec3f(0.0, 0.21, 0.21); // Powierzchnia tła
(#003737)
    let shadow_pigment = vec3f(0.0, 0.06, 0.06); // Podstawa cienia
(ciemniejszy barwnik tła)

    // Integracja światła otoczenia (ambient) i padającego cienia
    let final_color = mix(shadow_pigment, surface_color,
shadow_intensity * ui_params.ambient_intensity);

    // Nałożenie szumu przestrzennego (Atmospheric Noise) dla
maskowania bandingu
    let noise = fract(sin(dot(coord.xy, vec2f(12.9898, 78.233)))) *
43758.5453) * 0.02;

    return vec4f(final_color + noise, 1.0);
}

```

Kod ten demonstruje absolutną precyzję, przewyższając tysiącrotnie możliwości techniczne klasycznego modułu box-shadow silnika CSS, wprowadzając gładkie przejścia gradientowe na styku obiektu i tła.

Technika 3: Integracja Środowiskowa – Generic Sensor API w Cyklu Życia UI

Aby powyższy shader nie pozostał statycznym demem technologicznym, konieczne jest pompowanie do niego danych sprzętowych. Używamy w tym celu dwóch kluczowych interfejsów sprzętowych przeglądarek.

1. **Ambient Light Sensor API:** Zwraca wartość natężenia światła otoczenia w luksach (lux).
2. **DeviceOrientation API:** Zwraca kwaterniony opisujące fizyczne ułożenie urządzenia w przestrzeni 3D, z których wyliczamy wektor kierunkowy `light_direction` dla płaszczyzny 2D ekranu.

```
// lib/hardware/SensorySync.js
```

```
export class SpatialSensorySync {
  constructor(updateUniformCallback) {
    this.updateUniformCallback = updateUniformCallback;
    this.ambientIntensity = 1.0;
    this.lightDirection = { x: 0.5, y: -0.8 };

    this.initializeSensors();
  }

  initializeSensors([span_6] (start_span) [span_6] (end_span) [span_8] (start_sp
an) [span_8] (end_span) [span_10] (start_span) [span_10] (end_span) rs() {
    // Inicjalizacja detektora oświetlenia fizycznego
    [span_93] (start_span) [span_93] (end_span) [span_94] (start_span) [span_94]
(end_span)
    if ('AmbientLightSensor' in window) {
      try {
        const als = new AmbientLightSensor({ frequency: 10 });
        als.addEventListener('reading', () => {
          // Matematyczna normalizacja logarytmiczna
          (ludzkie oko reaguje nieliniowo)
          // Wynik w przedziale od 0 (absolutna ciemność) do
          30000+ (pełne nasłonecznienie)
          let normalized =
Math.l[span_82] (start_span) [span_82] (end_span) og10(als.illuminance +
1) / Math.log10(30000);
          this.ambientIntensity = Math.max(0.1,
Math.min(normalized, 1.0));
          this.dispatch();
        });
        als.start();
      } catch (err) {
        console.warn('Brak uprawnień czujnika ALS. Symulacja
włączona.');
```

```

(end_span)
    window.addEventListener('deviceorientation', (event) => {
        if (event.gamma === null || event.beta === null) return;

        // Konwersja kątów Eulera (beta, gamma) na wektor
        // Założenie: Płaszczyzna Z to wektor
        grawit[span_80](start_span)[span_80](end_span)acji. Kąt wychylenia
        kształtuje wirtualne słońce.
        const gammaRad = event.gamma * (Math.PI / 180);
        const betaRad = event.beta * (Math.PI / 180);

        this.lightDirection = {
            x: Math.sin(gammaRad),
            y: -Math.sin(betaRad) // Inwersja osi Y w grafice
            komputerowej
        };
        this.dispatch();
    });
}

dispatch() {
    // Przekazanie nowych zmiennych do buforów WebGPU
    this.updateUniformCallback({
        ambient_intensity: this.ambientIntensity,
        light_direction:
    });
}
}

```

Technika 4: Haptograficzna Matryca Osi Z (Sprzężenie Piezoelektryczne)

Zgodnie z wymaganiami architektury, każdy wyrenderowany i "wytloczony" element z użyciem cienia musi odpowiadać swojemu wyglądowi pod względem wagi przy interakcji dotykowej. Używamy ustandaryzowanego interfejsu Vibration API lub bibliotek takich jak Taptic Engine. Aby uniknąć chaosu strukturalnego, wprowadzamy globalny delegator zdarzeń, który odczytuje token głębi z właściwości semantycznych HTML i dopasowuje precyzyjny wzór milisekundowej częstotliwości sprzężenia.

```

// lib/haptics/ZAxisHaptics.js
[span_100](start_span)[span_100](end_span)[span_101](start_span)[span_
101](end_span)[span_102](start_span)[span_102](end_span)

// Baza sygnatur piezoelektrycznych zdefiniowanych w milisekundach
(czas działania i czas pauzy)
const Z_AXIS_HAPTIC_SIGNATURES = {

```

```

    'Z-0': , // Elementy płaskie (Base Background)
- delikatne wibracje niskotonowe
    'Z-1': , // Lekkie wypukłości (Raised Surface) -
standardowe kliknięcie
    'Z-2': , // Grube nawarstwienia (Overlay) -
wi[span_26](start_span)[span_26](end_span)elowarstwowe wrażenie
chropowatości
    'Z-3': // Modale powiadomień - gwałtowne uderzenie
(Pop-over)
};

export function attachSpatialHaptics(rootDOMElement) {
  if (!('vibrate' in navigator)) return;

  rootDOMElement.addEventListener('pointerdown', (e) => {
    const target = e.target.closest('[data-z-elevation]');
    if (target) {
      const elevation = target.getAttribute('data-z-elevation');
      const pattern = Z_AXIS_HAPTIC_SIGNATURES[elevation] ||
Z_AXIS_HAPTIC_SIGNATURES['Z-1'];

      // Wysłanie rozkazu do sprzętowego aktuatora z użyciem API
      navigator.vibrate(pattern);
    }
  });
}

```

Technika 5: CSS Houdini Bridge — Sklejenie Przestrzeni z DOM

Ostatnim ogniwem ekosystemu jest wstrzyknięcie technologii bezpośrednio w warstwę CSS. Nie każda aplikacja wymaga pełnoekranowego płótna (WebGL/WebGPU Canvas). Dla wielu zastosowań, lepszym modelem jest wykorzystanie CSS Paint API z modułu Houdini. Pozwala ono na wywołanie procesów obliczania wizualnego wprost z kodu CSS. Oznacza to, że tradycyjne kontenery i gridy układają układ strony, a wtyczki Houdini przejmują renderowanie tekstury tła.

Struktura składa się z pliku wtyczki rejestrującego nową metodę malowania :

```

// worklets/spatial-shadow.js (Zarejestrowane w PaintWorklet)
[span_106](start_span)[span_106](end_span)[span_107](start_span)[span_
107](end_span)
class SpatialShadowPainter {
  static get inputProperties() {
    return ['--z-elevation', '--light-angle'];
  }

  paint(ctx, size, properties) {
    // Kontekst malowania PaintWorklet (CanvasRenderingContext2D /
eksperymentalnie WebGPU)

```

```

[span_108] (start_span) [span_108] (end_span) [span_109] (start_span) [span_
109] (end_span)
    const zElevation =
parseInt(properties.get('--z-elevation').toString()) || 1;
    const lightAngle =
parseFloat(properties.get('--light-angle').toString()) || 45.0;

    // Tutaj w docelowym architekturze znajduje się logika
wstrzykiwania do silnika WebGPU
    // lub generowanie potężnych symulacji raymarchingu na
mniejszą skalę.[span_110] (start_span) [span_110] (end_span)
    // Ze względu na uwarunkowania bezpieczeństwa PaintWorklet
rysuje dynamiczne obrazy wektorowe:

    ctx.fillStyle = '#002121'; // Tło kaskadowe
    ctx.beginPath();
    // Generowanie zaokrąglenia i cienia kierunkowego
    ctx.roundRect(0, 0, size.width, size.height, 12);
    ctx.fill();

    //... (Złożona logika rzutowania SDF bezpośrednio na element
renderowany)
}
}
registerPaint('spatial-shadow', SpatialShadowPainter);

```

Wykorzystanie tego rozwiązania w kodzie aplikacji jest niezwykle eleganckie i zapewnia kompatybilność ze składnią CSS:

```

/* Wdrożenie w pliku stylów */
.futuristic-card {
    background-image: paint(spatial-shadow);
    --z-elevation: 2;
    --light-angle: calc(var(--hardware-gyro-angle) * 1deg);
    border-radius: 12px;
}

```

Podsumowanie i Implikacje Architektoniczne

Architektura interfejsów ewoluowała daleko poza to, co zaplanowali twórcy dokumentów HTML. Analiza zaprezentowana w tym badaniu dowodzi, że paradygmat maksymalizmu taktylnego nie może być zaspokojony poprzez zniekształcanie istniejących definicji CSS, takich jak statyczny, jednoosiowy box-shadow omijający fizykę światłocienia i właściwości sprzętowe. Narzędzia dzisiejszej epoki ugrzęzły w fundamentalnych ograniczeniach związanych ze ślepotą środowiskową, niewydajnością przeliczania klatek i strukturalnym ucinaniem geometrii układu. Zaprojektowanie "idealnej wersji" środowiska interaktywnego, ujętej w ramach koncepcji Holo-Haptic WebGPU Spatial Framework, stanowi radykalny przełom umożliwiający

zdominowanie branży. Wykorzystanie matematycznej bezbłędności Pól Odległości (SDF) w połączeniu ze sprzętowym akceleratom nowej generacji (WebGPU) transformuje interfejsy z płaskich obrazków w dynamiczne struktury świetlne, na które wpływ ma w czasie rzeczywistym żyroskop i oświetlenie pokoju. Haptograficzna Matryca osi Z ożywia wirtualną materię, dostarczając piezoelektrycznego uderzenia kompatybilnego z wagą obiektu. Jednoczesna identyfikacja natychmiastowych ulepszeń – technik takich jak *Double Wrapper*, wstrzymanie przeliczania rozmycia w oparciu o opacizy kompozytora, czy *Luminance Step-Up* – daje inżynierom oprogramowania niezbędny plan przejścia przez fazy przejściowe. Tylko organizacje inżynierskie, które zaakceptują wymóg neuroergonomii i zintegrują potoki cieniujące w całości uciekając z organicznego DOM do interfejsu WebGPU i modułów CSS Houdini, będą w stanie stworzyć oprogramowanie ostateczne – reagujące, fizycznie świadome i nieograniczone dotychczasowym płaskim dogmatem. Wdrożenie szczegółowo zaprezentowanego schematu środowiska produkcyjnego zapewni stabilną, zoptymalizowaną strukturę odporną na upływ czasu i zmiany rynkowe.

Cytowane prace

1. Latest Web Design Trends | Aufait UX, <https://www.aufaitux.com/blog/latest-web-design-trends/>
2. UI trends you will likely see in 2026 - Lummi, <https://www.lummi.ai/blog/ui-trends-2026>
3. GitHub - petronbot/ambient-light-sensor-demo: A prototype ponyfill for the `light-level` CSS `@media` feature, <https://github.com/petronbot/ambient-light-sensor-demo>
4. SensorEvent | API reference - Android Developers, <https://developer.android.com/reference/android/hardware/SensorEvent>
5. The Generic Sensor API - mobiForge, <https://mobiforge.com/design-development/the-generic-sensor-api>
6. Sensors and Local Interactions - Roadmap of Web Applications on Mobile - W3C, <https://www.w3.org/2019/04/web-roadmaps/mobile/sensors.html>
7. Signed distance field global illumination (SDFGI) - Godot Docs, https://docs.godotengine.org/en/stable/tutorials/3d/global_illumination/using_sdfgi.html
8. Signed Distance Fields for fun and profit - Showcase - three.js forum, <https://discourse.threejs.org/t/signed-distance-fields-for-fun-and-profit/77121>
9. Your first WebGPU app - Google Codelabs, <https://codelabs.developers.google.com/your-first-webgpu-app>
10. Chapter 34. Signed Distance Fields Using Single-Pass GPU Scan Conversion of Tetrahedra, <https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-34-signed-distance-fields-using-single-pass-gpu>
11. typegpu/sdf - Software Mansion, <https://docs.swmansion.com/TypeGPU/ecosystem/typegpu-sdf/>
12. GitHub - CedricGuillemet/SDF: Collection of resources (papers, links, discussions, shadertoy,...) related to Signed Distance Field, <https://github.com/CedricGuillemet/SDF>
13. How to Create a Liquid Raymarching Scene Using Three.js Shading Language | Codrops, <https://tympanus.net/codrops/2024/07/15/how-to-create-a-liquid-raymarching-scene-using-three-js-shading-language/>
14. Cocos Creator Implements Various Shader Effects Based On SDF, <https://forum.cocosengine.org/t/cocos-creator-implements-various-shader-effects-based-on-sdf/56106>
15. Screen space shadows (contact shadows) in WebGPU - Help appreciated! - Reddit, https://www.reddit.com/r/GraphicsProgramming/comments/1ee0g2t/screen_space_shadows_contact_shadows_in_webgpu/
16. Integrating Haptic Feedback for Image-based STEM Content within HTML and eTextBooks, <http://diagramcenter.org/integrating-haptic-feedback.html>
17. The Pulse of Interaction: Why Haptics and Vibration Define Modern UX - Medium,

<https://medium.com/@chamikasathsara1/the-pulse-of-interaction-why-haptics-and-vibration-define-modern-ux-1638dd1a9c9c> 18. Haptic Feedback Device Using 3D-Printed Flexible, Multilayered Piezoelectric Coating for In-Car Touchscreen Interface - MDPI, <https://www.mdpi.com/2072-666X/14/8/1553> 19. design Archives - Mediastreet, <https://mediastreet.ie/tag/design/> 20. Designing for Haptic Feedback: Enhancing User Interactions Through Touch - UX Pilot, <https://uxpilot.ai/blogs/enhancing-haptic-feedback-user-interactions> 21. CSS Houdini - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Properties_and_values_API/Houdini 22. CSS Houdini - Vincent De Oliveira, <https://iamvdo.me/en/blog/css-houdini> 23. The Structure of a WebGPU Renderer - Ryosuke, <https://whoisryosuke.com/blog/2025/structure-of-a-webgpu-renderer/> 24. React Native implementation of WebGPU using Dawn - GitHub, <https://github.com/wcandillon/react-native-webgpu> 25. WebGPU Shading Language - W3C, <https://www.w3.org/TR/WGSL/> 26. WebGL vs. WebGPU - FenixFox@Studios, https://fenixfox-studios.com/content/webgpu_vs_webgl/ 27. GPUCanvasContext: configure() method - Web APIs | MDN, <https://developer.mozilla.org/en-US/docs/Web/API/GPUCanvasContext/configure> 28. The Web's Sixth Sense: A Study of Scripts Accessing Smartphone Sensors, <https://sensor-js.xyz/webs-sixth-sense-ccs18.pdf> 29. Houdini APIs - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/Web/API/Houdini_APIs 30. WebGPU API - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API